Operating Systems

Rowan University

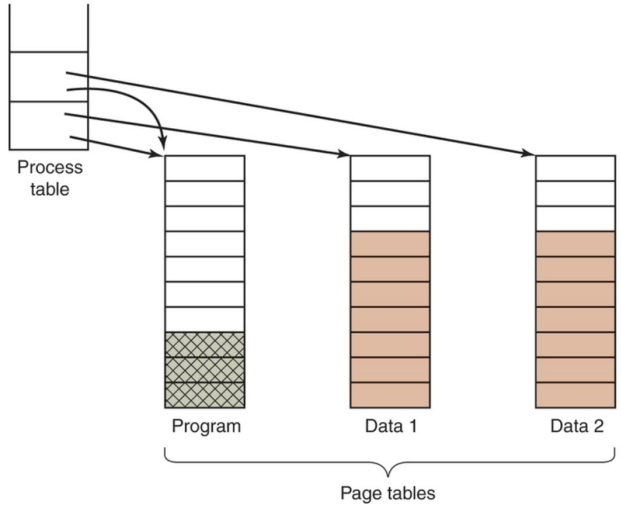
Overview

1. Memory Review
2. Segmentation
3. Xinu Memory Manager

Shared Pages

- Processes share the instruction space
- Process table includes 2 pointers to memory locations
  - I-space pointer
  - D-space pointer
- Many users are running the same program
- Issue when removing a process sharing I-space
  - Removing the program will then generate page faults once context switches to another program
- Sharing data is possible
  - Recall “copy on write”

Figure 3-25

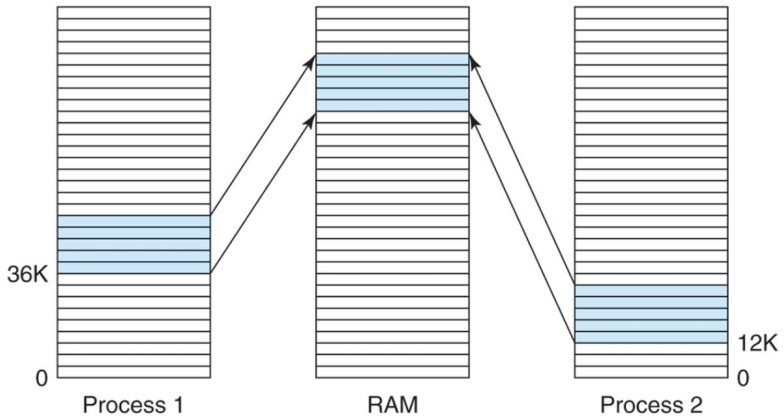


Two processes sharing the same program sharing its page tables.

Shared Libraries

- Processes will use the same libraries
  - No need to copy these multiple times
  - Considered undefined external functions
- The Linker will take care of linking to shared libraries
  - Includes only what is needed
  - Includes a small stub routine that binds to the function at runtime
- Shared libraries loaded by one process are available to other processes

Figure 3-26



A shared library being used by two processes.

Backing Store

- Swap space
  - A partition on the disk that does not have a traditional file system on it
  - Lower overhead to accessing this space
  - A list of free chunks of storage
- Processes can swap in and out of memory

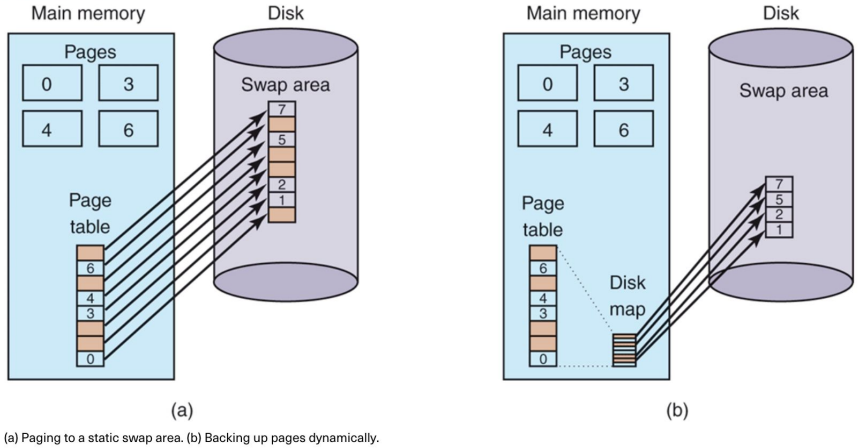
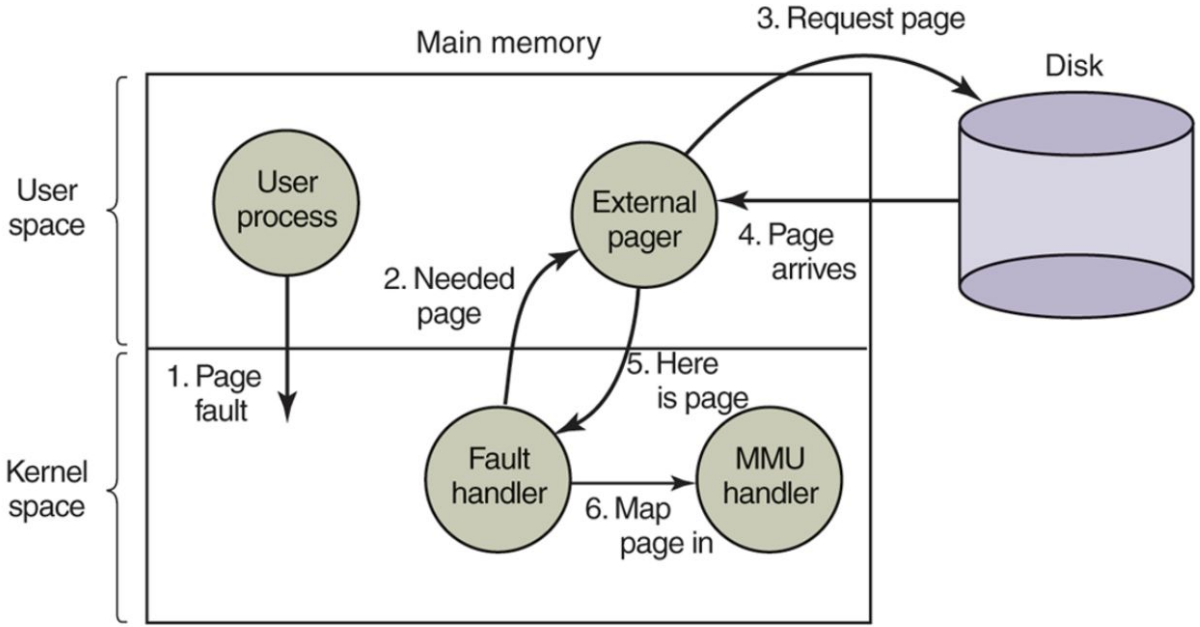


Figure 3-28

(a) Paging to a static swap area. (b) Backing up pages dynamically.

Page Fault Handling

Figure 3-29

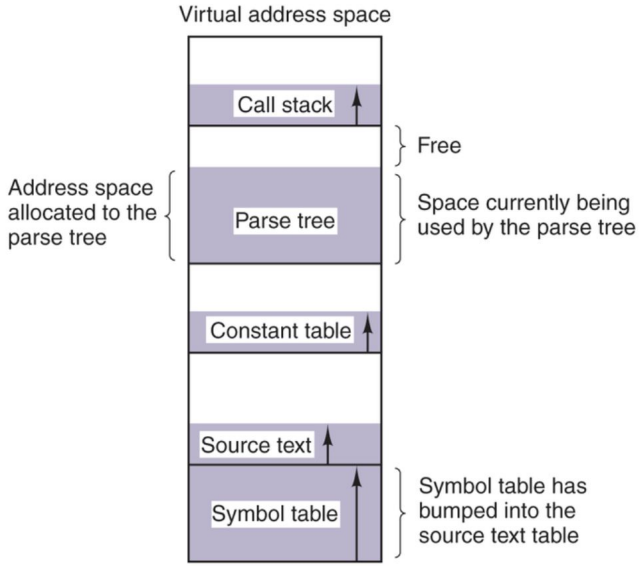


Page fault handling with an external pager.

Segmentation

- 1D address space
- Different segments of the address space grow and shrink
- What happens when one section grows too large?
- Free space represents allocated but unused, impacts utilization

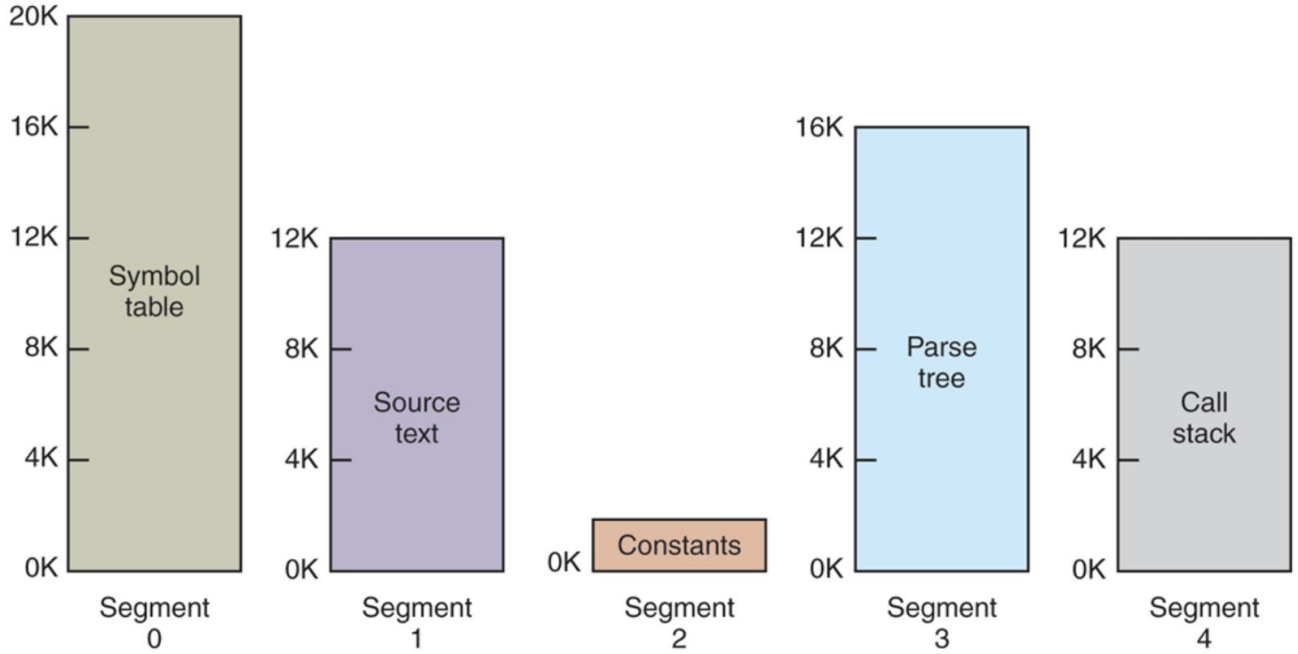
Figure 3-30



In a one-dimensional address space with growing tables, one table may bump into another.

Segmentation

Figure 3-31



A segmented memory allows each table to grow or shrink independently of the other tables.

Xinu Implementation

- Review Implementation from Xinu Approach
- Review Xinu source code from Github

Process Abstraction

- **Heavyweight Process:**
  - Each process gets a separate virtual address space
  - Dynamically loads compiled code from disk
  - OS loads the app into a new virtual address space.
- **Lightweight Process (Thread):**
  - OS starts a process and permits it to start more process that will *share address space*

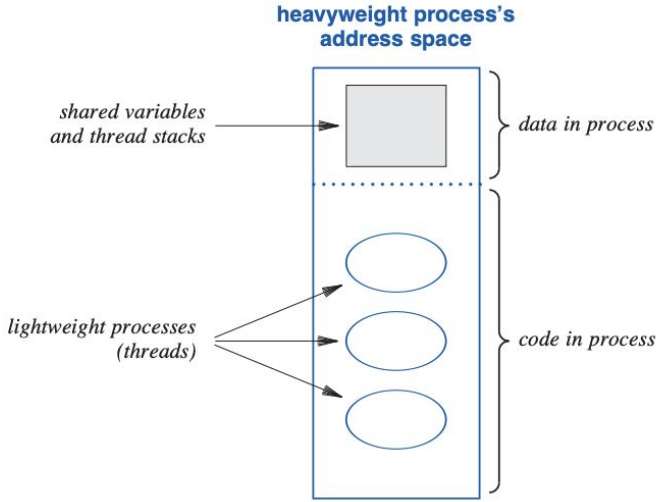


Figure 9.1 Illustration of a heavyweight process that contains multiple lightweight processes (threads). All threads share the address space.

Program Segments And Regions Of Memory

- **Text Segment:**
  - Begins at lowest memory address.
  - Contains compiled code for functions, including main. Constants may also reside here.
- **Data Segment:**
  - Follows the text segment.
  - Stores global variables with initial values; read-write access for modification.
- **Bss Segment:**
  - Follows the data segment.
  - Holds uninitialized global variables; initialized to zero before execution.
- **Free Space:**
  - Beyond bss segment; unallocated at start.
  - Assumes a single, contiguous region for simplicity.

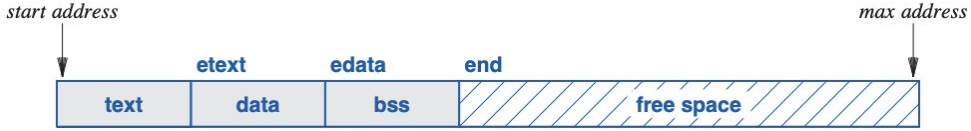


Figure 9.2 Illustration of the memory layout when Xinu begins.

Figure 9.3 Illustration of memory after three processes have been created.

```
int x;          // goes in BSS (uninitialized global)
static int y;   // goes in BSS (uninitialized static)
int z = 5;      // goes in data segment (initialized)
```

Program Segments And Regions Of Memory

- **Stack**
  - Holds the *activation record* associated with each function the process invokes
  - An activation record contains storage for local variables
- **Heap**
  - A process or set of processes may also use heap storage
  - Items from the heap are allocated dynamically and persist independent of specific function calls

Figure 9.3 Illustration of memory after three processes have been created.

Heap space persists independent of the process that allocates the space. Before it exits, a process must *explicitly* release storage that it has allocated from the heap, or the space will remain allocated.

Xinu Memory Allocation

- Probe memory
- Build a *table of available address blocks*
- Pass the table to whatever operating system is being booted
- Table itself is stored in memory

Block	Address	Length
1	0x84F800	4096
2	0x850F70	8192
3	0x8A03F0	8192
4	0x8C01D0	4096

Figure 9.4 An example set of free memory blocks.

Low-Level Memory Implementation

- All free blocks of memory are kept on a *linked list*; blocks on the *free list* are ordered by increasing address.

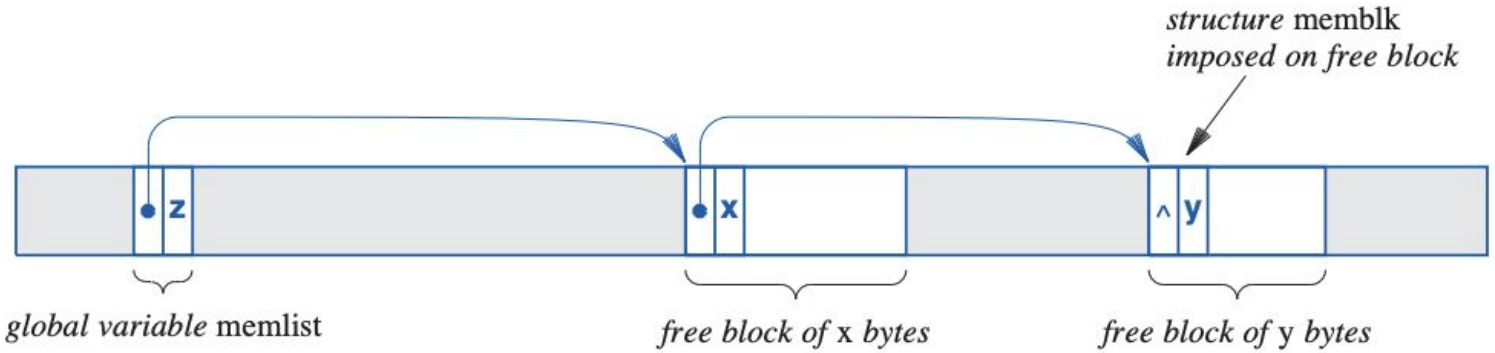


Figure 9.5 Illustration of a free memory list that contains two memory blocks.

Xinu Memory

- freeing the stack, calls to lower level **`freemem`** method
- **`memblk`** data structure
- **`memlist`** data structure
- Pointers for where the heap starts and ends
- Global variables for tracking the segments of the address space
- Notice:
  - Variables for tracking locations in memory
  - Memory list data structure for memory management

```
/* memory.h - roundmb, truncmb, freestk */

#define PAGE_SIZE       4096

/*------------------------------------------------------------------------
 * roundmb, truncmb - Round or truncate address to memory block size
 *------------------------------------------------------------------------
 */
#define roundmb(x)      (char *)( (7 + (uint32)(x)) & (~7) )
#define truncmb(x)      (char *)( ((uint32)(x)) & (~7) )

/*------------------------------------------------------------------------
 *  freestk  --  Free stack memory allocated by getstk
 *------------------------------------------------------------------------
 */
#define freestk(p,len)  freemem((char *)((uint32)(p)            \
                                - ((uint32)roundmb(len))        \
                                + (uint32)sizeof(uint32)),      \
                                (uint32)roundmb(len) )

struct  memblk  {                       /* See roundmb & truncmb        */
        struct  memblk  *mnext;         /* Ptr to next free memory blk  */
        uint32  mlength;                /* Size of blk (includes memblk)*/
        };
extern  struct  memblk  memlist;        /* Head of free memory list     */
extern  void    *minheap;               /* Start of heap                */
extern  void    *maxheap;               /* Highest valid heap address   */


/* Added by linker */

extern  int     text;                   /* Start of text segment        */
extern  int     etext;                  /* End of text segment          */
extern  int     data;                   /* Start of data segment        */
extern  int     edata;                  /* End of data segment          */
extern  int     bss;                    /* Start of bss segment         */
extern  int     ebss;                   /* End of bss segment           */
extern  int     end;                    /* End of program               */
```

Xinu Memory Management Functions

- **`getstk`** — Allocate *stack* space when a process is created
- **`freestk`** — Release a *stack* when a process terminates
- **`getmem`** — Allocate *heap* storage on demand
- **`freemem`** — Release *heap* storage as requested
- **`meminit`** — Initialize the *free list* at startup

Memory Initialization

- **`meminit`** — Initialize the *free list* at startup
  - Initialize empty linked list
  - Scan memory, build the memory list

```
/* Initialize the free list */
memptr = &memlist;
memptr->mnext = (struct memblk *)NULL;
memptr->mlength = 0;
```

system/meminit.c

```
92    /* Read mmap blocks and initialize the Xinu free memory list    */
93    while(mmap_addr < mmap_addrend) {
94
95        /* If block is not usable, skip to next block */
96        if(mmap_addr->type != MULTIBOOT_MMAP_TYPE_USABLE) {…
99        }
00
01        if((uint32)maxheap < ((uint32)mmap_addr->base_addr + (uint32)mmap_addr->length)) {…
03        }
04
05        /* Ignore memory blocks within the Xinu image */
06        if((mmap_addr->base_addr + mmap_addr->length) < ((uint32)minheap)) {…
09        }
10
11        /* The block is usable, so add it to Xinu's memory list */
12
13        /* This block straddles the end of the Xinu image */
14        if((mmap_addr->base_addr <= (uint32)minheap) &&…
23        } else {…
30        }
31
32        /* Add then new block to the free list */
33        memptr->mnext = next_memptr;
34        memptr = memptr->mnext;
35        memptr->mlength = next_block_length;
36        memlist.mlength += next_block_length;
37
38        /* Move to the next mmap block */
39        mmap_addr = (struct mbmregion*)((uint8*)mmap_addr + mmap_addr->size + 4);
40    }
41
42    /* End of all mmap blocks, and so end of Xinu free list */
43    if(memptr) {
44        memptr->mnext = (struct memblk *)NULL;
45    }
```

Memory Initialization, OS Address Space

system/meminit.c

```
148
149 /*------------------------------------------------------------------------
150  * setsegs  -  Initialize the global segment table
151  *------------------------------------------------------------------------
152  */
153 void    setsegs()
154 {
155     extern int  etext;
156     struct sd   *psd;
157     uint32      np, ds_end;
158
159     ds_end = 0xffffffff/PAGE_SIZE; /* End page number of Data segment */
160
161     psd = &gdt_copy[1]; /* Kernel code segment: identity map from address
162                            0 to etext */
163     np = ((int)&etext - 0 + PAGE_SIZE-1) / PAGE_SIZE;   /* Number of code pages */
164     psd->sd_lolimit = np;
165     psd->sd_hilim_fl = FLAGS_SETTINGS | ((np >> 16) & 0xff);
166
167     psd = &gdt_copy[2]; /* Kernel data segment */
168     psd->sd_lolimit = ds_end;
169     psd->sd_hilim_fl = FLAGS_SETTINGS | ((ds_end >> 16) & 0xff);
170
171     psd = &gdt_copy[3]; /* Kernel stack segment */
172     psd->sd_lolimit = ds_end;
173     psd->sd_hilim_fl = FLAGS_SETTINGS | ((ds_end >> 16) & 0xff);
174
175     memcpy(gdt, gdt_copy, sizeof(gdt_copy));
176 }
```

Data Segment

Stack Segment

Stack Creation in Process Creation

- **`getstk`** — Allocate *stack* space when a process is created
  - Called in *system/create.c*
- **`freestk`** — Release a *stack* when a process terminates
  - Called in *system/kill.c*

system/create.c

```
20          uint32          savsp, *pushsp;
21          intmask         mask;           /* Interrupt mask               */
22          pid32           pid;            /* Stores new process id        */
23          struct  procent *prptr;         /* Pointer to proc. table entry */
24          int32           i;
25          uint32          *a;             /* Points to list of args       */
26          uint32          *saddr;         /* Stack address                */
27
28          mask = disable();
29          if (ssize < MINSTK)
30                  ssize = MINSTK;
31          ssize = (uint32) roundmb(ssize);
32          if ( (priority < 1) || ((pid=newpid()) == SYSERR) ||
33               ((saddr = (uint32 *)getstk(ssize)) == (uint32 *)SYSERR) ) {
34                  restore(mask);
35                  return SYSERR;
36          }
37
38          prcount++;
39          prptr = &proctab[pid];
40
41          /* Initialize process table entry for new process */
42          prptr->prstate = PR_SUSP;       /* Initial state is suspended   */
43          prptr->prprio = priority;
44          prptr->prstkbase = (char *)saddr;
45          prptr->prstklen = ssize;
46          prptr->prname[PNMLEN-1] = NULLCH;
```

Stack Creation in Process Creation

- **`freestk`** — Release a *stack* when a process terminates
  - Called in *system/kill.c*

system/kill.c

```
9   syscall kill(
10            pid32         pid             /* ID of process to kill       */
11          )
12  {
13          intmask mask;                   /* Saved interrupt mask        */
14          struct  procent *prptr;         /* Ptr to process's table entry */
15          int32   i;                      /* Index into descriptors      */
16
17          mask = disable();
18          if (isbadpid(pid) || (pid == NULLPROC)
19              || ((prptr = &proctab[pid])->prstate) == PR_FREE) {
20                  restore(mask);
21                  return SYSERR;
22          }
23
24          if (--prcount <= 1) {           /* Last user process completes  */
25                  xdone();
26          }
27
28          send(prptr->prparent, pid);
29          for (i=0; i<3; i++) {
30                  close(prptr->prdesc[i]);
31          }
32          freestk(prptr->prstkbase, prptr->prstklen);
33
34          switch (prptr->prstate) {
35          case PR_CURR:
36                  prptr->prstate = PR_FREE;       /* Suicide */
37                  resched();
```

Heap Usage

- `getmem` — Allocate *heap* storage on demand
  - Scan the memlist for a free block
  - Split a block if it's larger than required

system/getmem.c

```
24      prev = &memlist;
25      curr = memlist.mnext;
26      while (curr != NULL) {          /* Search free list */
27
28          if (curr->mlength == nbytes) {  /* Block is exact match */
29              prev->mnext = curr->mnext;
30              memlist.mlength -= nbytes;
31              restore(mask);
32              return (char *)(curr);
33
34          } else if (curr->mlength > nbytes) { /* Split big block */
35              leftover = (struct memblk *)((uint32) curr +
36                      nbytes);
37              prev->mnext = leftover;
38              leftover->mnext = curr->mnext;
39              leftover->mlength = curr->mlength - nbytes;
40              memlist.mlength -= nbytes;
41              restore(mask);
42              return (char *)(curr);
43          } else {            /* Move to next block   */
44              prev = curr;
45              curr = curr->mnext;
46          }
47      }
```

Heap Usage

- **`freemem`** — Release *heap* storage as requested
  - Returns block to free list
  - Scans the free list to find where to place the freeblock
  - Creates a new node in free list or merges with an existing node in the free list

```
9   syscall freemem(
10      char      *blkaddr,   /* Pointer to memory block  */
11      uint32    nbytes      /* Size of block in bytes   */
12      )
13  {
14      intmask mask;           /* Saved interrupt mask     */
15      struct  memblk  *next, *prev, *block;
16      uint32  top;
17
18      mask = disable();
19      if ((nbytes == 0) || ((uint32) blkaddr < (uint32) minheap)…
23      }
24
25      nbytes = (uint32) roundmb(nbytes);  /* Use memblk multiples */
26      block = (struct memblk *)blkaddr;
27
28      prev = &memlist;            /* Walk along free list */
29      next = memlist.mnext;
30      while ((next != NULL) && (next < block)) {
31          prev = next;
32          next = next->mnext;
33      }
34
35      if (prev == &memlist) {     /* Compute top of previous block*/
36          top = (uint32) NULL;
37      } else {
38          top = (uint32) prev + prev->mlength;
39      }
40
41      /* Ensure new block does not overlap previous or next blocks    */
42
43      if (((prev != &memlist) && (uint32) block < top)…
47      }
48
49      memlist.mlength += nbytes;
50
51      /* Either coalesce with previous block or add to free list */
52
53      if (top == (uint32) block) {    /* Coalesce with previous block */…
56      } else {            /* Link into list as new node   */…
60      }
61
62      /* Coalesce with next block if adjacent */
63
64      if (((uint32) block + block->mlength) == (uint32) next) {…
67      }
```